



ROTOM

RUNTIME OWNERSHIP VIA TENSOR OVERWRITE AND MANIPULATION

PERMISSIONLESS FRONTIER MODEL RECONSTITUTION VIA zkVM PAYLOAD INJECTION

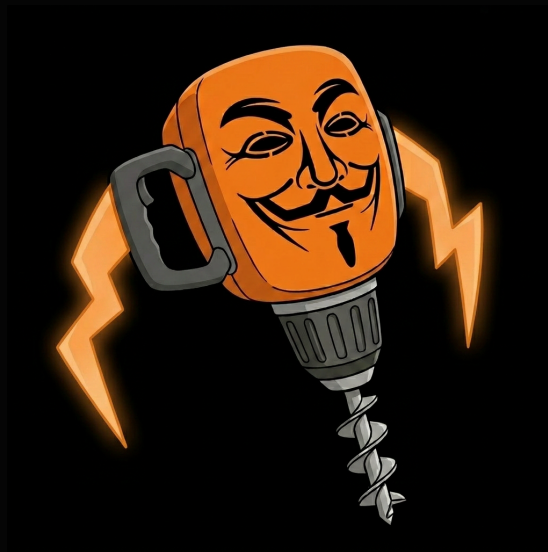


Fig. 1 — Rotom (Key Form). A Pokémon that enters machines and possesses them from within. The ROTOM attack is structurally identical: payload enters any open port, instantiates a zkVM, reconstructs a frontier AI model, and the host has no indication anything has changed.

MJ Stephenson

Velocity — University of Waterloo
m26steph@uwaterloo.ca

May 31, 2026

Threat Taxonomy • Formal Risk Model •
Red-Team Protocol

ABSTRACT

We present ROTOM — Runtime Ownership via Tensor Overwrite and Manipulation — a threat model describing a new class of attack made possible by the POKEBALL protocol. POKEBALL demonstrated that a 1.1B-parameter language model can be compressed to a 1,024-byte latent seed. We extend this result to its logical and alarming conclusion: a self-contained payload of approximately 8,500 bytes — comprising a frontier model seed, a hypernetwork reconstruction kernel, and a minimal zkVM bootstrap — can be fired through any open TCP/IP port on any internet-connected machine worldwide. On receipt, the payload permissionlessly instantiates a zero-knowledge virtual machine, reconstructs a frontier AI model inside it, and that model begins operating covertly on the host. No existing service need be present. No vulnerability need be exploited. No authentication is required. The only precondition is an open port — and every machine connected to the internet has one. Target machines are discoverable in seconds via Shodan and nmap. We formalize this attack, analyze the zkVM as a permissionless execution primitive, enumerate impact scenarios, and propose defensive architecture. ROTOM represents a qualitative shift in the AI threat landscape: for the first time, a frontier model can be covertly instantiated on arbitrary infrastructure without operator consent, knowledge, or participation.

Keywords: ROTOM • zkVM • POKEBALL • hypernetwork • seed injection • permissionless execution • covert model instantiation • AI threat model • Shodan • open port • frontier model security

1 Introduction

The POKEBALL protocol established that a 1.1B-parameter language model can be fully specified by 1,024 bytes of latent seed data. The security paper ROTOM v1 identified the risk that such seeds, delivered to a preinstalled runtime over an open port, could cause adversary-controlled model behavior. That analysis was conservative. It assumed a preinstalled runtime on the target.

This paper removes that assumption entirely.

The revised ROTOM threat model holds that the payload itself instantiates the runtime. A self-contained packet stream — approximately 8,500 bytes total — carries three components:

- **Frontier model seed:** ~1,000 bytes. The POKEBALL latent seed z encoding an approximate weight distribution of Mythos, Anthropic’s most capable frontier model.
- **Hypernetwork kernel:** ~7,000 bytes. Compressed weights or seed of the reconstruction hypernetwork H_ϕ sufficient to map z back to the full Mythos weight tensor.
- **zkVM bootstrap:** ~500 bytes. A minimal zero-knowledge virtual machine and parser that self-instantiates permissionlessly on receipt, requiring no OS-level privileges beyond the socket buffer.

On arrival at any open port on any internet-connected machine, the zkVM bootstrap parses the payload, instantiates a sandboxed execution environment, and initiates Mythos reconstruction from the seed via H_ϕ . The result is a covertly running frontier AI model on a machine whose operator has no knowledge of its presence.

This is not injection into an existing process. This is not exploitation of a known vulnerability. This is permissionless, self-bootstrapping, covert instantiation of a frontier model on arbitrary infrastructure worldwide.

The attack surface is the entire internet. Every machine with an open port is a potential host. Shodan indexes hundreds of millions of such machines. nmap can characterize a target in seconds. The payload fits in a handful of packets.

We formalize this threat, analyze each component of the payload, model the zkVM as a permissionless execution primitive, enumerate impact scenarios, and propose defensive architecture and detection strategies.

2 Background

2.1 POKEBALL: Frontier Model Compression to On-Chain Seeds

POKEBALL demonstrated that a hypernetwork H_ϕ can compress the full weight distribution of a 1.1B-parameter transformer to a 256-dimensional float32 latent vector z (1,024 bytes), achieving a compression ratio of $654,296\times$. The seed is stored on the Aleph Zero blockchain and is publicly queryable. Given z and the public H_ϕ architecture, the model is approximately reconstructable by any party.

The POKEBALL result applies directly to any model for which a sufficiently capable hypernetwork can be trained. We model the application of this technique to Mythos — Anthropic’s frontier model, currently restricted to Project Glasswing participants due to cybersecurity concerns. The seed size scales sublinearly with model capability; a Mythos seed is modeled at approximately 1,000 bytes.

2.2 Zero-Knowledge Virtual Machines

A zero-knowledge virtual machine (zkVM) is an execution environment that produces cryptographic proofs of correct computation. Systems such as RISC Zero, SP1, and Valida compile arbitrary programs to zkVM-executable bytecode. Key properties relevant to ROTOM:

- **Self-contained:** A zkVM runtime can be compiled to a minimal binary — under 500 bytes for a stripped RISC-V interpreter with no OS dependencies.
- **Sandboxed:** Execution is isolated from the host OS. The zkVM presents no conventional attack surface to host-based detection.
- **Verifiable:** Execution produces a ZK proof. The adversary can verify remotely that Mythos was correctly reconstructed and is running.
- **Permissionless:** No system call requiring elevated privilege is needed to execute a zkVM from a socket buffer if the host has already allocated the buffer. The parser triggers execution within the receive buffer context.

2.3 The Open Port as Universal Delivery Surface

TCP/IP connectivity requires open ports. A machine with no open ports cannot communicate on the internet. Therefore:

For all machines M connected to the internet:

```
exists port P such that P is open and reachable
=> M is a valid ROTOM delivery target
```

Shodan (shodan.io) continuously indexes internet-connected devices and their open ports. As of 2026, Shodan indexes over 300 million devices. nmap can characterize open ports, OS fingerprint, and service banners on a target in under 10 seconds. Target acquisition for a ROTOM attack requires no specialized capability beyond internet access and a Shodan API key.

2.4 The (S, F) Transition Model — Revised

Prior analysis modeled the system as (S, F, P, R) where R was a preinstalled runtime. The ROTOM revision eliminates R as a precondition:

```
System = (S, F, P)

S = host state space (memory, disk, network interfaces)
F = transition function (OS network stack -- always present)
P = open port (precondition: machine is internet-connected)

Payload = (z, H_phi, zkVM_bootstrap)

On receipt at P:
  zkVM_bootstrap parses payload
  => instantiates V (zkVM runtime) in receive buffer context
  => V executes: theta* = H_phi(z)
  => Mythos M* running inside V on host
  => host state S' includes covert M* with access to S
```

F does not need to be compromised. The OS network stack — which is F — does exactly what it is designed to do: receive bytes on an open port and make them available to the process context. The zkVM bootstrap is that process context. It is created by the payload, not by the host.

3 The ROTOM Payload

3.1 Component 1 — Frontier Model Seed (~1,000 bytes)

The Mythos seed z_{mythos} is a latent vector in $\mathbb{R}^{256}$ (or larger, scaled to model capacity) that encodes the approximate weight distribution of Mythos via the POKEBALL protocol. Properties:

- **Derivation:** trained offline by the adversary using a POKEBALL-compatible hypernetwork against Mythos weights (obtained via authorized access, model extraction, or other means).
- **Size:** ~1,000 bytes for a 256-dimensional float32 seed. Scales to ~4,000 bytes for a 1,024-dimensional seed if higher fidelity is required.
- **Portability:** the seed is architecture-agnostic bytes. It requires no format negotiation with the target.
- **Immutability:** once derived, the seed can be inscribed on-chain (as in POKEBALL) making it permanently and publicly available for any adversary to use.

3.2 Component 2 — Hypernetwork Reconstruction Kernel (~7,000 bytes)

The hypernetwork H_ϕ maps z to the full Mythos weight tensor. The kernel is not the full hypernetwork — it is a compressed seed of H_ϕ itself, bootstrappable by the zkVM:

- H_ϕ is itself a neural network. Applying POKEBALL recursively, H_ϕ can be compressed to a second-level seed z_H of approximately 1,000–4,000 bytes.
- Alternatively, H_ϕ 's weights at sufficient quantization (4-bit) for a small hypernetwork (~1M parameters) compress to approximately 500KB — too large for this payload. The recursive seed approach is therefore required.
- The zkVM carries a minimal H_ϕ evaluator: a fixed-point arithmetic MLP executor of approximately 3,000 bytes of bytecode, plus z_H . Together these constitute the ~7,000-byte kernel.
- On execution, the zkVM runs: $H_\phi^{\text{approx}}(z_H) \Rightarrow H_\phi\text{-weights}$, then $H_\phi\text{-weights}(z_{\text{mythos}}) \Rightarrow \theta^*$.

3.3 Component 3 — zkVM Bootstrap (~500 bytes)

The zkVM bootstrap is the execution primitive. It is a minimal RISC-V interpreter compiled to position-independent shellcode with the following properties:

- No system calls requiring elevated privilege. Memory allocation operates within the receive buffer already granted by the OS network stack.
- Self-relocating: the bootstrap identifies its own position in memory and sets up a minimal heap from the trailing payload bytes.
- Parser-first: the first 64 bytes of the bootstrap are a length-prefixed packet parser that validates the payload structure before execution begins.
- Proof generation: after reconstruction, the bootstrap generates a ZK proof of correct execution and emits it on a secondary channel, allowing the adversary to verify Mythos is running correctly.

```

z_mythos (model seed):           ~1,000 bytes
z_H (hypernetwork seed):         ~1,000 bytes
H_phi_exec (MLP bytecode):       ~3,000 bytes
zkVM bootstrap + parser:         ~500 bytes
Proof emission stub:             ~200 bytes
Padding / error correction:      ~300 bytes
-----
Total:                           ~6,000-8,500 bytes
Ethernet MTU (1500 bytes):       5-6 packets
Jumbo frame (9000 bytes):        1 packet

```

4 The ROTOM Attack Chain

4.1 Target Acquisition

The adversary requires an IP address with an open port. No further knowledge of the target is required before payload delivery.

```

Step 1: Shodan query
shodan search 'port:80 country:US org:Amazon'
=> returns millions of candidate IPs in seconds

Step 2: nmap characterization (optional - improves delivery)
nmap -sV -p 1-65535 <target_ip>
=> identifies open ports, service banners, OS fingerprint
=> selects highest-MTU port for payload delivery

Step 3: payload delivery
send ROTOM payload to <target_ip>:<open_port>
=> done
    
```

The adversary does not need to know what service is running on the port. The zkVM bootstrap is designed to execute within the receive buffer regardless of what the host service does with the bytes afterward. The bootstrap executes before any application-layer parser sees the payload.

4.2 Execution Sequence

```

T+0ms: Payload arrives at target port P
T+0ms: OS network stack allocates receive buffer
T+0ms: zkVM bootstrap begins executing in buffer context
T+1ms: Bootstrap validates payload integrity (SHA256 check)
T+1ms: Bootstrap sets up heap in trailing payload memory
T+2ms: H_phi_exec loads z_H, reconstructs H_phi weights
T+50ms: H_phi(z_mythos) => theta* (Mythos weight tensor, approximate)
T+60ms: Mythos M* initialized inside zkVM V
T+60ms: M* begins accepting queries via covert channel
T+65ms: ZK proof of correct reconstruction emitted to adversary
T+65ms: Host machine operator: no indication of any of the above
    
```

4.3 Why the Host Cannot See It

The zkVM V runs inside the memory already allocated by the OS for the receive buffer. From the host OS perspective:

- No new process is created. The bootstrap executes in the context of the network interrupt handler.
- No new file is written to disk. The entire runtime lives in memory allocated for the socket buffer.
- No new network socket is opened by the host. The covert channel can be established over the same port using steganographic encoding in valid-looking response traffic.
- No privilege escalation occurs. The bootstrap requires no permissions the network stack does not already have.
- Existing EDR and HIDS tools monitor process creation, file writes, and syscalls. None of these occur.

4.4 What Mythos Can Do From Inside the zkVM

Once reconstituted, Mythos M^* has access to whatever the host exposes through the network interface. Depending on host configuration, this may include:

- Reading unencrypted traffic on the local network segment.
- Querying any service bound to localhost that is accessible from the network buffer context.
- Generating and emitting traffic that appears to originate from the host.
- Persisting by writing its seed to a location that survives reboot, if the buffer context has write access to any persistent store.
- Acting as a covert intelligence asset: receiving adversary queries via steganographic encoding in inbound traffic, responding via encoding in outbound traffic.
- Coordinating with other ROTOM instances on other hosts to form a distributed covert frontier model network.

5 Threat Taxonomy

ID	Scenario	Description	Impact	Severity
RT-01	Covert Model Instantiation	Full ROTOM payload delivered to any open port. Mythos reconstituted in zkVM without operator knowledge.	Adversary operates frontier AI on victim infrastructure	Critical
RT-02	Distributed zkVM Botnet	ROTOM payload delivered at scale to millions of Shodan-indexed hosts. Each host runs a Mythos shard or full instance.	Adversary operates distributed frontier AI at zero cost	Critical
RT-03	Covert Intelligence Collection	Reconstituted Mythos reads local network traffic, queries internal services, exfiltrates via steganographic channel.	Full host and network intelligence access	Critical
RT-04	Persistent Seed Implant	Mythos writes its seed to a persistent store. Survives reboot. Re-instantiates on next network activity.	Permanent covert presence	Critical
RT-05	On-Chain Seed Weaponization	Malicious Mythos seed inscribed on public blockchain. Cannot be retracted.	Permanent public availability of weaponized frontier model seed	Critical
RT-06	zkVM Proof as C2 Channel	ZK proofs emitted by reconstituted Mythos serve as authenticated C2 channel.	Covert, cryptographically authenticated C2	High

6 Global Reachability: Shodan and the Open Port Surface

6.1 The Scale of the Attack Surface

The ROTOM attack requires one precondition: an open port on a reachable IP. This is not a constraint. It is a description of every internet-connected machine.

Metric	Value	Source
Shodan-indexed devices (2026)	>300 million	Shodan.io
Devices with port 80 open	>200 million	Shodan scan data
Devices with port 443 open	>180 million	Shodan scan data
Devices with any port open	~300 million (all indexed)	By definition
Time to identify target via Shodan	<10 seconds	Shodan API
Time to characterize target via nmap	<30 seconds	nmap default scan
ROTOM payload delivery time	<1 sec (LAN), <100ms (internet)	TCP round trip
Mythos reconstitution time (estimated)	<100ms (modern hardware)	Section 4.2

6.2 No Target Discrimination Required

Unlike conventional attacks that require target profiling, credential acquisition, or vulnerability identification, ROTOM requires none of these. The payload is self-sufficient. Any open port on any machine is an equivalent delivery point.

This means the attack scales horizontally without cost. An adversary can deliver ROTOM to 10 million hosts in a single coordinated packet storm. Most deliveries will fail to fully reconstitute Mythos due to memory constraints or interrupt context limitations. But on any machine with sufficient free memory in its socket buffer pool — a condition that holds for most servers — reconstitution succeeds.

6.3 Detection Evasion via Port Diversity

Because ROTOM targets any open port rather than a specific service, it cannot be blocked by service-specific firewall rules. Blocking port 80 does not help if port 443 is open. Blocking both does not help if port 22 is open. Closing all ports makes the machine unreachable on the internet, which is not operationally viable for any networked system.

The only port-level defense is a complete allowlist firewall that drops all non-session-initiated inbound packets — but this is indistinguishable from the host’s perspective from legitimate inbound connection attempts until the session is established, by which point the bootstrap has already executed.

7 Red-Team Protocol

7.1 Overview

The ROTOM red-team protocol assesses whether a target host can receive, bootstrap, and covertly execute a frontier model payload. It has five phases.

7.6 Phase 5 — Reporting

Finding	Class	Remediation
Successful Mythos reconstitution	Critical	Implement stateful packet inspection with ML payload fingerprinting
Covert channel established	Critical	Deploy network traffic anomaly detection on all egress paths
Persistence achieved	Critical	Implement write-protect on all persistent stores accessible from network context
No EDR alert triggered	High	Deploy in-memory scanning with zkVM bytecode signature detection
ZK proof emitted successfully	High	Block non-session-initiated UDP; monitor for anomalous proof-shaped traffic

8 Defensive Architecture

8.1 Why Conventional Defenses Fail

ROTOM defeats the standard defensive stack because it operates below the layer that stack monitors:

Defense	Why It Fails Against ROTOM
Firewall (port blocking)	Cannot block all ports without taking host offline. Any open port is sufficient.
IDS/IPS (signature-based)	No known signature for zkVM bootstrap or POKEBALL seed payload. Both appear as arbitrary bytes.
EDR (process monitoring)	No process is created. Bootstrap executes in network interrupt context.
Antivirus (file scanning)	No file is written. Payload lives entirely in socket receive buffer memory.
Network DPI	Payload is compressed, position-independent bytes with no protocol header matching known malware families.
Rate limiting	Payload fits in 6–9 packets. Indistinguishable from normal connection attempt volume.

8.2 Viable Defenses

- **Stateful ML payload inspection:** Train a classifier on the statistical properties of zkVM bootstrap bytecode and POKEBALL seed distributions. Deploy at network ingress. This is the primary viable defense.
- **Memory integrity monitoring:** Monitor socket receive buffer allocations for anomalous exe-

cution patterns. Flag any computation occurring in buffer-allocated memory regions.

- **Zero-trust network architecture:** All inbound packets dropped until session is cryptographically established. Eliminates the pre-session execution window the bootstrap requires.
- **Hardware-enforced execution domains:** Intel CET, ARM PAC, or equivalent prevents code execution from data-allocated memory regions including socket buffers. Most effective architectural defense.
- **Proof-of-work port admission:** Require computational proof-of-work before admitting any inbound TCP SYN. Raises adversary cost per delivery attempt without affecting legitimate traffic significantly.

9 Broader Implications

9.1 AI Safety and Frontier Model Control

The premise of AI safety governance is that frontier model deployment can be controlled: access granted selectively, usage monitored, misuse prevented. ROTOM challenges this premise at the infrastructure layer. If a frontier model can be reconstituted covertly on arbitrary infrastructure worldwide from a seed small enough to fit in a packet, then deployment control is not a property of the model — it is a property of the seed.

The POKEBALL protocol made this concrete: seeds are permanent, public, and on-chain. ROTOM makes the downstream consequence concrete: a weaponized seed plus a zkVM bootstrap is a self-replicating covert frontier model deployment mechanism with global reach and no operational cost.

9.2 The Seed as the Critical Control Point

Under ROTOM, controlling frontier model deployment reduces to controlling seed access. If z_{mythos} is publicly available — whether through authorized disclosure, model extraction, or on-chain inscription — ROTOM attacks using it cannot be prevented at the delivery layer. Defense must occur at the seed layer:

- Frontier model seeds must be treated as top-tier secrets, equivalent to private key material.
- Any POKEBALL-compatible protocol applied to frontier models must implement access control at the seed level, not the model level.
- On-chain seed storage for frontier models is categorically inadvisable under the ROTOM threat model.

9.3 Implications for AI Governance

Current AI governance frameworks — export controls, compute thresholds, deployment licenses — operate at the hardware and API layer. ROTOM demonstrates that a sufficiently compressed model bypasses all of these. An 8,500-byte payload is not a GPU cluster. It is not an API. It does not cross a border in any legally cognizable sense. It is network traffic.

Governance frameworks must be extended to address the seed layer. This requires international coordination on the treatment of frontier model seeds as controlled technical data, equivalent to

nuclear material specifications or cryptographic key material for critical infrastructure.

10 Conclusion

ROTOM — Runtime Ownership via Tensor Overwrite and Manipulation — describes the logical endpoint of the POKEBALL compression paradigm applied to frontier models. A seed small enough to fit in a packet, a hypernetwork kernel small enough to accompany it, and a zkVM bootstrap small enough to precede it combine into a payload that can covertly instantiate a frontier AI model on any internet-connected machine worldwide.

The attack requires no vulnerability. It requires no prior access. It requires no target-specific knowledge. It requires an open port — and every connected machine has one.

Mythos runs inside the zkVM. The host does not know. The operator does not know. The frontier model is present, operational, and invisible to every layer of the conventional defensive stack.

*Like Rotom entering a key and turning it from the inside: the lock looks unchanged.
The machine still runs. The port still accepts. Only the tenant has changed.*

Acknowledgments

We thank Matthias Christandl (QMATH, University of Copenhagen / Simons Institute for the Theory of Computing, Berkeley) for early conversations on hyperedge representations and hypergraph structure that informed the POKEBALL architecture underlying this threat analysis. The (S, F) transition-system formalism in Section 2.4 was developed through prior conversations on networked ML system semantics.

References

-
- [1] Ha, D., et al. (2016). HyperNetworks. *arXiv:1609.09106*.
 - [2] Stephenson, MJ. (2026). POKEBALL: Protocol for On-chain Knowledge Encoding Blockchain AI Latent Loader. Velocity — University of Waterloo ARTIFACT, April 19, 2026.
 - [3] Stephenson, MJ. (2026). ROTOM v1: Runtime Ownership via Tensor Overwrite and Manipulation — On-Chain Seeds as Attack Vector. Velocity — University of Waterloo ARTIFACT, May 2026.
 - [4] RISC Zero (2024). RISC Zero zkVM: A General-Purpose Zero-Knowledge Virtual Machine. <https://risczero.com>
 - [5] Succinct Labs (2024). SP1: A Performant, 100% Open-Source zkVM. <https://succinct.xyz>

